VLM Reliability Toolkit

Presenter script: how to explain the application flow, and what to show in the live demo

Part	Content	Time
A	One-minute pitch (the problem and our idea)	1 min
B	Application flow, step by step, with what to say	3 min
C	Live demo run sheet (exact inputs, where to click, what to point at)	5 min
D	Benchmarks (POPE and CHAIR) talk track	1–2 min
E	Likely questions and short answers	as needed
F	Checklist before the demo, and fallbacks	—

How to read this script

- Text in a **purple SAY box** is meant to be spoken. Say it in your own words; do not read it word for word.
- Plain text is stage direction: what to click, what to point at, and why.
- Suggested speakers (change freely): **Prithvi**, pitch and flow; **Kavyansh**, live demo; **Harsh**, live model and re-ranking; **Yash**, benchmarks and questions.

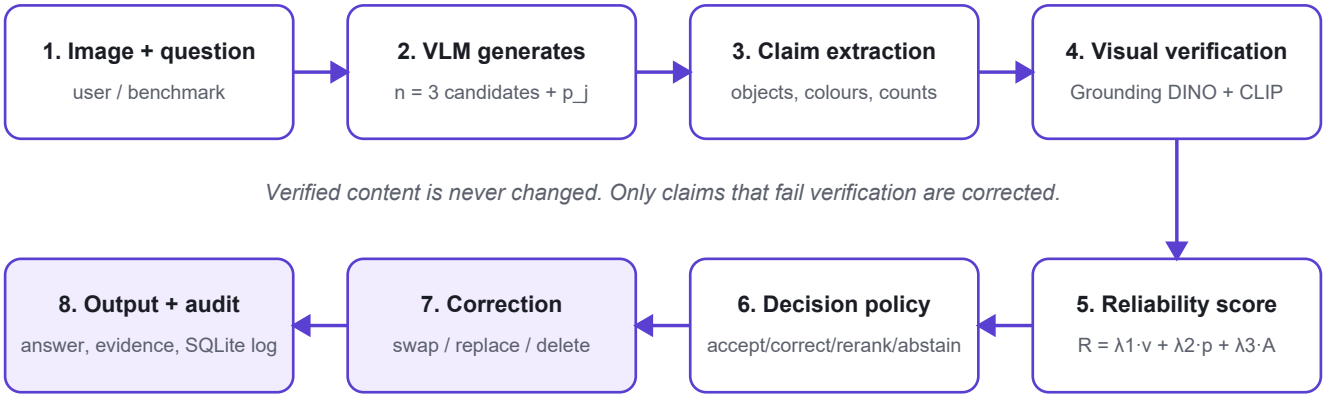
A. One-minute pitch

SAY — Prithvi: Vision-language models such as LLaVA or Gemini describe images fluently, but they make two kinds of mistakes. First, they **hallucinate objects**: they mention a surfboard that is not in the picture. Second, they **mis-bind attributes**: they say *blue shirt and red jeans* when the shirt is red and the jeans are blue. Retraining the model to fix this is expensive and has to be repeated for every model.

SAY: Our toolkit is a **reliability layer that sits after any VLM**. It needs no training and works with any model. It breaks the answer into small checkable claims, checks each claim against the image with separate vision models, scores how reliable each answer is, and then accepts it, corrects it, picks a better candidate, or abstains. Anything it cannot verify is flagged as uncertain instead of being guessed, and everything is logged.

B. Application flow

Draw or show this diagram first, then walk through the eight steps.



1. Input

The user uploads an image and asks a question, for example *Describe this image*. In the benchmarks, the image and question come from the POPE or CHAIR dataset.

> **SAY:** The input is an image plus a question. It can come from a person using our dashboard, or from a benchmark.

In the code: Dashboard (Next.js) → FastAPI /api/analyze

2. Generate n candidates

The VLM writes three answers: one greedy (the normal answer) and two sampled at temperature 0.7. For local models we also record p_j, the model's own confidence: the average token probability.

> **SAY:** We don't trust a single answer. We ask the model for three. The first is exactly what the model would normally say, so it is our baseline. The other two give us alternatives and let us measure agreement.

In the code: vlm.py (Gemini, OpenAI, Groq), models/hf_vlm.py (LLaVA on GPU)

3. Claim extraction

Each answer is split into atomic claims: existence claims (*there is a dog*), binding claims (*the shirt is red*), counts (*two dogs*) and spatial relations.

> **SAY:** A sentence can be partly right and partly wrong, so we check it claim by claim. *A man in a red shirt holding a surfboard* becomes: man exists, shirt exists, shirt is red, surfboard exists.

In the code: claims/extractor.py, using the 80 COCO object classes and their synonyms

4. Visual verification

Existence: Grounding DINO searches the image for the object. Its best box confidence is the score (paper Eq. 1). Binding: we crop that object's box and ask CLIP how likely the colour or material is *inside that crop*, multiplied by the detection confidence (Eq. 2).

> **SAY:** This is the key idea. We check the colour on the object's **own crop**, not on the whole image. That way, a blue sky can't make *blue shirt* look correct. That is how we catch swapped attributes.

In the code: verification/grounding.py, attribute_verifier.py, service.py

5. Reliability score

Each candidate gets $R = \lambda_1 \cdot$(mean claim score) + $\lambda_2 \cdot p_j$ + $\lambda_3 \cdot$(agreement with the other candidates), with weights 0.6 / 0.2 / 0.2 (Eq. 3). If a term is unavailable, for example p_j for an API model, its weight is shared across the other terms.

> **SAY:** Each answer gets one reliability number between 0 and 1. It combines three signals: how well its claims are verified, how confident the model was, and how much the three answers agree with each other.

In the code: reliability/engine.py

6. Decision policy

Per claim: score ≥ τ_{hi} → ACCEPT; score ≤ τ_{lo} → CORRECT; in between → ABSTAIN (flag as uncertain). Per answer: if the top answer's R is below 0.5 and another candidate passes, RERANK to that candidate. The thresholds are tuned on a held-out set, never on the test data.

> **SAY:** Each claim gets one of four actions. Accept means the claim is clearly true. Correct means it is clearly false and we know the fix. Abstain means we are not sure, so we don't guess; we flag it and leave it unchanged. Re-rank means another candidate is clearly more reliable, so we use that one instead.

In the code: core/policy.py, core/pipeline.py

7. Correction

Edits only the failed spans: it swaps mis-bound attributes (red ↔ blue), replaces a wrong colour with the verified one, and deletes hallucinated objects together with their phrase.

> **SAY:** We make the smallest possible edit. If a claim is verified, its words are never touched.

In the code: correction/engine.py

8. Output and audit log

The final answer is returned with every claim, score, box and action. Each run is saved with the image hash, so any run can be reopened on the Experiments page.

> **SAY:** Everything is traceable. For each answer you can see which claim was checked, the evidence, the score, and what we did. Uncertain claims are flagged rather than guessed.

In the code: database/db.py (SQLite), Experiments page in the dashboard

C. Live demo run sheet (about 5 minutes)

Open **http://127.0.0.1:3000/analyze**. All three images were run once beforehand, so each scene returns in a few seconds. Images are in **test_images/**; the file names start with Gemini_Generated_Image_ followed by the code shown below.

Scene 1 — Attribute correction and object deletion (90 s)

- Upload **…7nkn…png**: a man in a red T-shirt walking down a street.
- Open **Controlled candidate override** and paste:
  A man in a blue shirt walks down the street holding a surfboard.
- Click **Run reliability analysis** (about 5 s).
- Point at the **Evidence canvas**: boxes for the person and the shirt; no box for a surfboard.
- Point at the **claims table**: person ACCEPT · shirt ACCEPT · shirt is blue CONTRADICTED → CORRECT (replace blue → red), with the shirt crop · surfboard UNSUPPORTED → CORRECT (delete).
- Point at the **Correction diff**: *A man in a red shirt walks down the street.*

> **SAY — Kavyansh:** We typed a deliberately wrong caption. The toolkit found the shirt, checked the colour inside the shirt's own crop, where red scores 0.59 and blue 0.05, and replaced the colour. It found no surfboard, so it deleted that phrase. The rest of the sentence is untouched. This is the paper's Table IV behaviour on a real image.

Scene 2 — A live model (60 s)

- Clear the override. Upload **…8jkx…png** (busy city street) and keep the question *Describe the objects and their colors.* Gemini 2.5 Flash writes three candidates live (about 20 s).
- Expected: cars, yellow taxis, trees and people are ACCEPTed; a few claims such as *blue sedans* or *buildings* are flagged as uncertain and left unchanged.
- If the Wi-Fi is slow, skip this scene; scenes 1 and 3 need no API.

> **SAY — Harsh:** A good model is mostly right, and the toolkit says so: verified claims pass through unchanged. Where the image evidence is weak it does not guess; it flags the claim instead.

Scene 3 — Re-ranking (60 s)

- Upload **…dqp7…png** (dog, cat and an old car in a park). In the override, paste three candidates separated by **|||**:
  A golden retriever dog and a white cat sit on the grass near a green car and a red bicycle. ||| *A dog and a cat sit near a car.* ||| *A brown dog sits on a bench next to two cats and a blue umbrella.*
- Decision panel: **RERANK** — *Top candidate R=0.49 < 0.50; candidate-2 has R=0.86.*
- Open the **Candidates** page: the chart and the three cards (R 87% / 49% / 46%).
- Reliability panel: mean v 82%, agreement 100%, p_j n/a (API model).

> **SAY — Harsh:** The first answer invents a bicycle and gets details wrong, so its reliability is below 0.5. The second answer is simpler but fully verified and agrees with the others, so the toolkit selects it. That is the re-rank rule from the paper.

Scene 4 — Benchmarks (60–90 s)

- Open the **Evaluation** page and use the talk track in part D.

D. Benchmarks talk track

SAY — Yash: To measure this at scale, we ran LLaVA-1.5-7B on two standard hallucination benchmarks on free Kaggle GPUs. **POPE** asks yes-or-no questions such as *Is there a dog in the image?* over 450 COCO images, in three difficulty splits: random, popular and adversarial. **CHAIR** asks the model to caption 500 images and counts how many mentioned objects are not really there.

Metric	What it means	Better
POPE accuracy / F1	Correct yes/no answers about object presence	Higher
POPE yes-ratio	How often the model says yes. Hallucinating models say yes too often.	Closer to 50%
$CHAIR_S$	Share of captions with at least one hallucinated object	Lower
$CHAIR_I$	Share of mentioned objects that are hallucinated	Lower
Object recall	Real objects still mentioned. Checks we didn't over-delete.	Higher
Latency ratio	Toolkit time ÷ base model time	Lower

SAY: Compare the **base VLM** row with the **full toolkit** row. The ablation rows switch off one part at a time (attribute binding, the reliability score, or multiple candidates) to show what each part contributes. Thresholds were tuned on 50 held-out images that are never in the test set. One honest note: POPE and CHAIR only measure **object** hallucination. The attribute-binding correction is what you saw in the live demo.

Measured results (LLaVA-1.5-7B, Kaggle T4, thresholds tuned on the held-out split, λ = 0.6 / 0.2 / 0.2)

CHAIR (500 images)	CHAIR_S ↓	CHAIR_I ↓	Recall ↑	Latency
Base VLM	63.8	29.6	76.2	1.0×
Full toolkit	**57.2**	**25.5**	75.4	2.67×
No attribute binding	57.6	26.0	75.5	2.67×
No reliability scoring	59.6	27.2	75.8	2.67×
n = 1 (single candidate)	59.6	27.2	75.8	1.23×

POPE (2,700 per split)	Precision base → toolkit	F1 base → toolkit	Yes % base → toolkit
Random	96.9 → 97.8	86.2 → 86.3	40.0 → 39.4
Popular	94.7 → 95.1	85.3 → 85.2	41.0 → 40.6
Adversarial	89.6 → 90.3	83.2 → 83.2	43.3 → 42.7

SAY — Yash: On CHAIR the toolkit cuts hallucinated captions from 63.8 to 57.2 percent and hallucinated objects from 29.6 to 25.5 percent, while object recall drops by less than one point. Removing re-ranking or using a single candidate gives back about a third of that gain. On POPE, LLaVA is already conservative, so the gain is small: precision goes up on every split and the yes-ratio goes down, with F1 flat.

E. Likely questions

Q: Why not just fine-tune the model?

Fine-tuning is costly and has to be redone for every model and version. Our layer is training-free and model-agnostic. We use the same code for Gemini, OpenAI and LLaVA.

Q: Isn't the verifier also a model that can be wrong?

Yes. That's why we don't trust any single score. We use per-type thresholds tuned on held-out data and abstain in the uncertain band instead of guessing. Grounding DINO and CLIP are trained for detection and matching, which is a narrower and more reliable task than open-ended generation.

Q: Why three candidates?

Agreement between independent samples is a strong signal of reliability, and a second candidate gives us something to re-rank to. The *n = 1* ablation shows what is lost without it.

Q: Why crop before checking colour?

If you ask CLIP about the whole image, any blue in the scene can support *blue shirt*. Checking inside the object's box ties the attribute to that object. That is the binding problem.

Q: How much slower is it?

We report the latency ratio in the benchmark table. Verification results are cached per image, so repeated analysis and ablations add almost no extra cost.

Q: What can't it do?

It doesn't fix reasoning errors, small or hidden objects the detector misses, or out-of-vocabulary objects. Claim-extraction errors carry through, and colour depends on lighting. These are listed as limitations in the paper.

Q: Is p_j available for Gemini?

No. Gemini doesn't return token probabilities, so that term is marked n/a and its weight is shared across verification and agreement. For LLaVA in the benchmarks, p_j is computed from the real token probabilities.

Q: Why did it not correct every colour?

It only corrects when the crop gives positive evidence for a different colour. For example, CLIP reads faded denim as grey or brown rather than blue, so a claim about jeans colour is flagged, not changed. Abstaining there is the intended behaviour.

Q: Why keep the paper weights instead of the tuned ones?

On POPE, answers are one yes/no token, so the model's token confidence is trivially well calibrated and an unconstrained search puts all weight on it. That would make R ignore the image on captions. With the paper weights the full toolkit reaches CHAIR_S 57.2 instead of 59.4.

Q: Where do the thresholds come from?

From evaluation/tune.py on a separate 50-image held-out POPE split: τ_{lo} maximises F1, τ_{hi} targets 95% precision for accepted claims. They are saved to results/tuned.yaml.

F. Checklist before the demo

- Backend and dashboard running: http://127.0.0.1:8010/health returns ok, and http://127.0.0.1:3000/analyze loads. If not: powershell -ExecutionPolicy Bypass -File scripts\dev.ps1, wait 10 s, refresh.
- Evaluation page shows the POPE and CHAIR tables.

- The three demo images are ready in test_images/ (codes 7nkn, 8jkx, dqp7).
- Copy the two override captions from this script into a notes file so they can be pasted quickly.
- Close other heavy apps: verification runs on the CPU on this laptop.

If something breaks

Problem	What to do
No Wi-Fi or Gemini fails	Skip scene 2. Scenes 1 and 3 use the candidate override and need no API key.
Dashboard won't start	Run python demo_paper_example.py. It prints the Table IV trace with no models and no network.
Laptop fails	Switch to the MacBook backup.